Universidad de Carabobo
Facultad de Ciencias y Tecnología
Computación

Práctica de Laboratorio N° 4

Arquitectura del Computador – Interrupciones y E/S en MIPS32

Profesor:
José Canache

Estudiante:
Gabriel Vargas
C.I: 32.035.567

Valencia, Octubre 2026

Cuestionario y Desarrollo del Informe

1. Ciclo de una Interrupción de Hardware

El ciclo de atención a una interrupción de hardware transcurre en las siguientes etapas clave:

1. **Generación del evento:** El dispositivo periférico (teclado, reloj, etc.) activa una línea de interrupción física (INT) hacia el procesador.
2. **Detección:** Al finalizar el ciclo de instrucción actual, la CPU detecta la línea de interrupción activa si las interrupciones están habilitadas.
3. **Salvado de estado inicial y redirección:** La CPU guarda la dirección de la siguiente instrucción en el registro `EPC` (Exception Program Counter), deshabilita interrupciones en el registro `Status` y salta a la rutina de servicio (`0x80000180`).
4. **Atención por Software (ISR):** La rutina guarda los registros en la pila, atiende al periférico y restaura los datos.
5. **Retorno:** La instrucción `eret` restaura el estado y vuelve a la instrucción guardada en `EPC`.

2. Cuadro Comparativo: Gestiones de E/S (Sondeo vs Interrupciones)

Criterio	**E/S con Sondeo (Polling)**	**E/S con Interrupciones**
Mecanismo	La CPU consulta en bucle el registro de estado del periférico.	El periférico notifica a la CPU mediante una señal cuando está listo.
Uso de CPU	Ineficiente; desperdicia ciclos de reloj en espera activa.	Eficiente; la CPU ejecuta tareas hasta ser interrumpida.
Latencia	Depende de la frecuencia de consulta en el bucle.	Mínima; respuesta inmediata tras terminar la instrucción actual.
Implementación	Simple, resuelta por software.	Requiere hardware específico y rutina de servicio (ISR).

3. Ventajas de las Interrupciones

El uso de interrupciones maximiza el rendimiento del procesador al eliminar la espera activa (polling). Permite ejecutar tareas secundarias mientras los periféricos procesan datos de manera asíncrona.

4. Registros Especiales en MIPS32 (Coprocesador 0)

- `$12 (Status)`: Controla la habilitación global e individual de interrupciones.
- `$13 (Cause)`: Registra la causa de la excepción o la interrupción activa.
- `$14 (EPC)`: Contiene la dirección de la instrucción interrumpida para retornar tras la rutina.
- `$8 (BadVAddr)`: Registra la dirección de memoria que produjo una falla de direccionamiento.

5. Importancia de Salvar el Contexto

Es necesario guardar los registros generales en la pila al entrar a una rutina de servicio (ISR) porque la interrupción ocurre de forma asíncrona. Si la ISR modifica los registros sin restaurarlos, alterará el estado del programa principal al regresar, causando errores de ejecución o corrupción de memoria.

6. Situaciones de Excepción y Pipeline

a) **4 Situaciones comunes:**

- Desbordamiento aritmético (`Overflow`).
- Error de alineación o dirección no válida (`Address Error`).
- Código de instrucción no válido (`Reserved Instruction`).
- División entre cero.

b) **Etapas del Pipeline:**

- **IF (Instruction Fetch):** Dirección no alineada al buscar la instrucción.
- **ID (Instruction Decode):** Instrucción ilegal o no reconocida.
- **EX (Execute):** Overflow aritmético o división por cero.
- **MEM (Memory Access):** Acceso a dirección inválida o no alineada en memoria.

7. Tratamiento de Excepciones e Interrupciones

a) **Diferencias:** Las interrupciones son **asíncronas** (generadas externamente por dispositivos de E/S). Las excepciones son **síncronas** (generadas internamente por la ejecución de una instrucción específica).

b) **Redirección y EPC:** Al ocurrir una excepción, la CPU redirige la ejecución a la dirección vectorial `0x80000180`. El registro `EPC` guarda la dirección de retorno para reanudar el programa.

8. Habilitación de Interrupciones

a) **Habilitación:**

- **Teclado/Pantalla:** Bit 1 en el registro de control del dispositivo en MMIO (`0xffff0000`).
- **Procesador:** Bit 0 (`IE` - Interrupt Enable) y bits de máscara de interrupción (`IM`) en el registro `Status` del Coprocesador 0.

b) **Consecuencia:** Las señales de los dispositivos no serán atendidas por la CPU; el hardware notificará la solicitud pero el procesador las ignorará.

9. Procesamiento de Interrupción de Reloj

a) **Pasos:**

- Ocurre el pulso del reloj, el hardware deshabilita interrupciones, guarda el PC en `EPC` y salta a `0x80000180`.
- La ISR guarda el contexto, procesa la actualización de tiempo, restaura los registros guardados y ejecuta `eret`.

b) **Importancia:** Garantiza la transparencia total del sistema operativo y evita la perdida de datos del programa interrumpido.

10. Control de Ejecución con Temporizador

a) Una interrupción de reloj permite al sistema operativo quitar periódicamente el control a un programa, evitando bucles infinitos y terminando procesos que excedan el límite de tiempo.

b) El programa debe cancelar la alarma o temporizador registrado mediante una llamada al sistema antes de salir.

11. Análisis y Discusión de Resultados

Se verificó el funcionamiento de los programas en la herramienta *Keyboard and Display MMIO Simulator* de MARS. Se demostró que la temporización mediante lecturas del reloj del sistema y el filtrado en registros MMIO permite simular correctamente el comportamiento de periféricos y semáforos controlados por eventos asíncronos en MIPS32.